Database Design: ExplainIt V1

Oct 19, 2026 · @Kaushtubh Mishra

Overview

ExplainIt V1 stores everything except media in one Supabase Postgres database on the free tier: six tables, four of them owned row by row by a user, and nine functions that keep the important updates atomic.

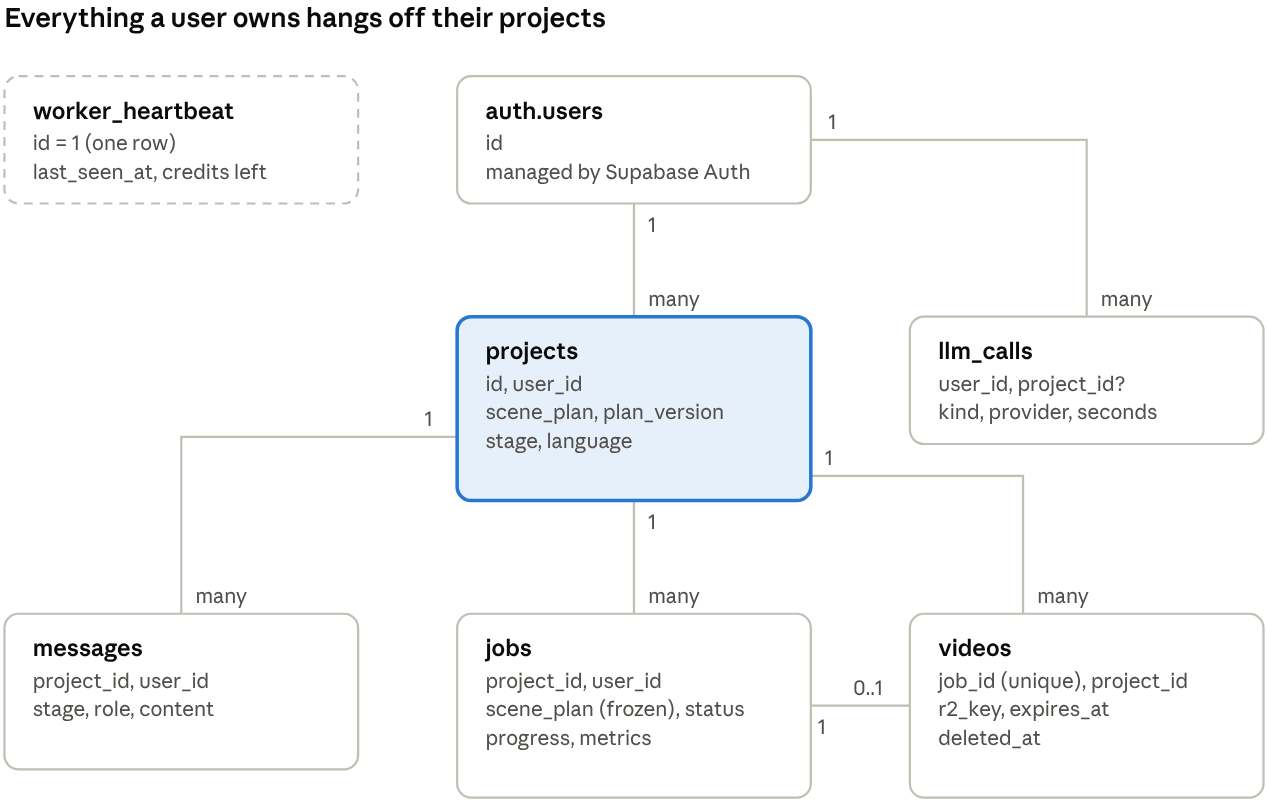
Table	Holds	Written by
`projects`	One video idea: input, language, scene plan, plan version, stage	API
`messages`	Brainstorm and revision chat turns	API
`jobs`	One render request, with a frozen copy of the plan, status, and progress	API creates; worker updates
`videos`	One finished MP4 in R2 and its expiry	Worker
`llm_calls`	One row per Gemini call, for rate limits and cost tracking	API
`worker_heartbeat`	A single row the laptop worker refreshes every 30 s	Worker

Users come from Supabase Auth (`auth.users`); deleting a user deletes everything they own. Audio, timings, and MP4 files never go in the database: they live on the laptop's D: drive and in Cloudflare R2.

This design adds three things to the V1 design doc: the `llm_calls` and `worker_heartbeat` tables, and a frozen plan copy on each job, so a render always uses exactly the plan that was approved.

Entity relationships

A user owns projects, and nearly everything else belongs to a project; only `worker_heartbeat` stands alone.



Entity relationships · 6 tables plus auth.users

Each job produces at most one video, and a project can have many of both, since every re-render is a new job. `llm_calls` also points to its project when there is one, and keeps the row (with the link cleared) if the project is deleted, so usage counts stay correct.

Tables and columns

The whole schema is one migration file, `supabase/migrations/0001_init.sql`; fixed lists of values are Postgres enums, so a typo cannot reach the database.

```
create type input_mode    as enum ('idea', 'notes', 'brainstorm');
create type project_stage as enum ('planning', 'storyboard', 'rendering',
'ready');
create type chat_stage    as enum ('brainstorm', 'storyboard', 'video');
create type chat_role     as enum ('user', 'assistant');
create type job_status    as enum ('queued', 'voicing', 'rendering',
'uploading', 'done', 'failed', 'cancelled');
create type llm_kind      as enum ('brainstorm', 'script', 'revise');

create table projects (
```

```
  id            uuid primary key default gen_random_uuid(),
  user_id       uuid not null references auth.users (id) on delete cascade,
  title         text not null default 'Untitled' check (char_length(title) <=
70),
  language      text not null check (language in ('en-IN', 'hi-IN')),
  duration_target smallint not null default 60 check (duration_target in (60,
90)),  -- seconds, chosen by the user
  theme         text not null default 'auto' check (theme in ('auto', 'sky',
'forest', 'sunset', 'night')),  -- 'auto' lets Gemini pick
  input_mode    input_mode not null,
  input_text    text not null default '' check (char_length(input_text) <=
5000),
  scene_plan    jsonb,                          -- null until the first
script
  plan_version  integer not null default 0,     -- +1 on every saved plan
  stage         project_stage not null default 'planning',
  created_at    timestamptz not null default now(),
  updated_at    timestamptz not null default now()
);

create table messages (
  id          uuid primary key default gen_random_uuid(),
  project_id  uuid not null references projects (id) on delete cascade,
  user_id     uuid not null references auth.users (id) on delete cascade,
  stage       chat_stage not null,
  role        chat_role not null,
  content     text not null check (char_length(content) <= 4000),
  created_at  timestamptz not null default now()
);

create table jobs (
  id                uuid primary key default gen_random_uuid(),
  project_id        uuid not null references projects (id) on delete
cascade,
  user_id           uuid not null references auth.users (id) on delete
cascade,
  plan_version      integer not null,
  scene_plan        jsonb not null,              -- frozen copy rendered by
the worker
  voice             text not null check (voice in ('elevenlabs', 'edge')),
  status            job_status not null default 'queued',
  progress          smallint not null default 0 check (progress between 0
and 100),
```

```
  changed_scene_ids  text[] not null default '{}',
  attempts           smallint not null default 0,
  error              text,
  metrics            jsonb not null default '{}',
  created_at         timestamptz not null default now(),
  claimed_at         timestamptz,
  finished_at        timestamptz
);

create table videos (
  id                 uuid primary key default gen_random_uuid(),
  project_id         uuid not null references projects (id) on delete cascade,
  user_id            uuid not null references auth.users (id) on delete
cascade,
  job_id             uuid not null unique references jobs (id) on delete
cascade,
  r2_key             text not null unique,
  size_bytes         bigint not null,
  duration_seconds   numeric(5, 1) not null,
  expires_at         timestamptz not null,           -- 23:59 IST on the
render day
  deleted_at         timestamptz,                    -- set when removed from
R2
  created_at         timestamptz not null default now()
);

create table llm_calls (
  id          bigint generated always as identity primary key,
  user_id     uuid not null references auth.users (id) on delete cascade,
  project_id  uuid references projects (id) on delete set null,
  kind        llm_kind not null,
  provider    text not null,                     -- e.g. gemini:gemini-flash-
latest
  ok          boolean not null,
  attempts    smallint not null default 1,
  seconds     numeric(6, 1) not null,
  created_at  timestamptz not null default now()
);

create table worker_heartbeat (
  id                   smallint primary key default 1 check (id = 1),  --
exactly one row
```

```
  last_seen_at        timestamptz not null,
  voice_credits_left  integer,
  worker_version      text
);
```

Every table has `user_id` where a user owns rows, even when it could be found through `project_id`. That keeps each security policy a one-line check and makes per-user counts fast.

Indexes and constraints

Each index serves one known query, and one partial unique index enforces the most important business rule: a project can have only one active render.

```
-- one active render per project
create unique index jobs_one_active_per_project on jobs (project_id)
  where status in ('queued', 'voicing', 'rendering', 'uploading');

create index projects_library     on projects (user_id, updated_at desc);
create index messages_by_project  on messages (project_id, stage,
created_at);
create index jobs_queue           on jobs (created_at) where status =
'queued';
create index jobs_by_project      on jobs (project_id, created_at desc);
create index jobs_by_user_day     on jobs (user_id, created_at);
create index videos_by_project    on videos (project_id, created_at desc);
create index videos_to_expire     on videos (expires_at) where deleted_at is
null;
create index llm_calls_by_user    on llm_calls (user_id, created_at);

-- keep updated_at honest
create function set_updated_at() returns trigger language plpgsql as $$
begin new.updated_at := now(); return new; end $$;
create trigger projects_updated_at before update on projects
  for each row execute function set_updated_at();
```

Index	Serves
`jobs_one_active_per_project`	Render button; a second render gets `409 job_active` from the insert itself

Index	Serves
`projects_library`	The library page, newest first
`messages_by_project`	Loading one stage's chat history
`jobs_queue`	`claim_next_job()` finding the oldest queued job
`jobs_by_project`	The project's latest and active job
`jobs_by_user_day`, `llm_calls_by_user`	Rate-limit counts
`videos_to_expire`	The day-end cleanup

Row-level security

RLS is on for every table, and each policy is the same check: the row's `user_id` is the signed-in user. The worker uses the service role key, which bypasses RLS, so only it can change job progress or create videos.

```
alter table projects         enable row level security;
alter table messages         enable row level security;
alter table jobs             enable row level security;
alter table videos           enable row level security;
alter table llm_calls        enable row level security;
alter table worker_heartbeat enable row level security;

-- projects: full control of your own
create policy own_projects on projects for all to authenticated
  using (user_id = auth.uid()) with check (user_id = auth.uid());

-- messages and llm_calls: read and add your own
create policy read_own_messages on messages for select to authenticated using
(user_id = auth.uid());
create policy add_own_messages  on messages for insert to authenticated with
check (user_id = auth.uid());
create policy read_own_llm      on llm_calls for select to authenticated
using (user_id = auth.uid());
create policy add_own_llm       on llm_calls for insert to authenticated with
check (user_id = auth.uid());
```

```
-- jobs: read your own; create only queued jobs; no direct updates
create policy read_own_jobs on jobs for select to authenticated using
(user_id = auth.uid());
create policy add_own_jobs  on jobs for insert to authenticated
  with check (user_id = auth.uid() and status = 'queued' and progress = 0);
-- cancel and retry go through cancel_job() and retry_job() (see Database
functions)

-- videos: read your own; the worker writes them
create policy read_own_videos on videos for select to authenticated using
(user_id = auth.uid());

-- heartbeat: any signed-in user may read it
create policy read_heartbeat on worker_heartbeat for select to authenticated
using (true);
```

Table	Signed-in user can	Worker (service role)
`projects`	Read, create, edit, delete own	Set `stage`
`messages`	Read and add own	None
`jobs`	Read own; add queued; cancel or retry through functions	Claim, update progress, finish, fail
`videos`	Read own	Create, mark deleted
`llm_calls`	Read and add own	None
`worker_heartbeat`	Read	Update

Users have no update policy on `jobs` at all. The two things they may do to a job, cancel it while `queued` or retry it after `failed`, go through two small functions that check ownership and status and change nothing else, so progress or results can never be faked from the browser.

Database functions

Every multi-step change is one function, so it either fully happens or does not happen at all; three are called by the API as the user, and six only by the worker.

Function	Called by	Does
`save_plan(project, plan, expected_version)`	API, as the user	Saves the plan only if the version still matches; bumps the version; raises `version_conflict` otherwise
`cancel_job(job)`	API, as the user	`queued` to `cancelled`, for the owner only
`retry_job(job)`	API, as the user	`failed` to `queued`, for the owner, if no other job is active
`claim_next_job()`	Worker	Takes the oldest queued job, safely even if two workers ever run
`complete_job(job, r2_key, size, duration, metrics)`	Worker	Finishes the job, creates the video row with tonight's expiry, sets the project to `ready`
`fail_job(job, error, metrics)`	Worker	Marks the job failed and returns the project to `storyboard`
`requeue_stale_jobs(minutes)`	Worker, on start	Requeues jobs stuck mid-render, for example after the laptop slept
`worker_heartbeat(credits, version)`	Worker, every 30 s	Updates the single heartbeat row
`ist_day_end()`	Other functions	23:59 today in India time, as a UTC timestamp

```
create function ist_day_end() returns timestamptz language sql stable as $$
  select (date_trunc('day', now() at time zone 'Asia/Kolkata') + interval '23
hours 59 minutes')
         at time zone 'Asia/Kolkata' $$;

create function save_plan(p_project uuid, p_plan jsonb, p_expected integer)
returns integer language plpgsql as $$            -- runs as the user, so RLS
applies
declare v integer;
begin
```

```
  update projects
     set scene_plan = p_plan, plan_version = plan_version + 1,
         stage = case when stage = 'planning' then 'storyboard' else stage
end
   where id = p_project and plan_version = p_expected
  returning plan_version into v;
  if v is null then raise exception 'version_conflict'; end if;
  return v;
end $$;

create function claim_next_job() returns setof jobs language sql as $$
  update jobs set status = 'voicing', claimed_at = now(), attempts = attempts
+ 1
   where id = (select id from jobs where status = 'queued'
               order by created_at limit 1 for update skip locked)
  returning * $$;

create function complete_job(p_job uuid, p_key text, p_size bigint,
p_duration numeric, p_metrics jsonb)
returns uuid language plpgsql as $$
declare j jobs; v uuid;
begin
  update jobs set status = 'done', progress = 100, finished_at = now(),
metrics = metrics || p_metrics
   where id = p_job returning * into j;
  insert into videos (project_id, user_id, job_id, r2_key, size_bytes,
duration_seconds, expires_at)
  values (j.project_id, j.user_id, j.id, p_key, p_size, p_duration,
ist_day_end())
  returning id into v;
  update projects set stage = 'ready' where id = j.project_id;
  return v;
end $$;

create function requeue_stale_jobs(p_minutes integer default 30) returns
integer language plpgsql as $$
declare n integer;
begin
  update jobs set status = 'queued'
   where status in ('voicing', 'rendering', 'uploading')
     and claimed_at < now() - make_interval(mins => p_minutes);
  get diagnostics n = row_count;
  return n;
```

```
end $$;

create function cancel_job(p_job uuid) returns void language plpgsql security
definer
set search_path = public as $$
begin
  update jobs set status = 'cancelled', finished_at = now()
   where id = p_job and user_id = auth.uid() and status = 'queued';
  if not found then raise exception 'not_cancellable'; end if;
end $$;

-- worker-only functions are not callable from the browser
revoke execute on function claim_next_job, complete_job, fail_job,
requeue_stale_jobs, worker_heartbeat
  from public, anon, authenticated;
```

`fail_job`, `retry_job`, and `worker_heartbeat` follow the same patterns: one guarded update each, with `retry_job` checked against `auth.uid()` like `cancel_job`, and `worker_heartbeat` an insert that updates the single row on conflict.

JSON columns

Three columns store JSON; their shape is owned by the Zod schemas in `packages/shared`, not by the database, so the API validates them before every write.

Column	Shape	Size
`projects.scene_plan`	`ScenePlan` from `schema.ts`: version, title, language, theme, 5–10 scenes	About 3–6 KB
`jobs.scene_plan`	The same shape, copied when the job is created and never edited	About 3–6 KB
`jobs.metrics`	Filled in by the worker as it goes (below)	Under 1 KB

```
{ "voice": { "provider": "elevenlabs", "voiceId": "…", "charactersVoiced":
1180,
              "scenesReused": 5, "seconds": 21 },
  "render": { "frames": 2215, "seconds": 158, "concurrency": 2,
"cueTimedAnimations": "17/17" },
  "upload": { "bytes": 14200000, "seconds": 6 },
  "videoSeconds": 73.8 }
```

The database stores these as `jsonb` and never queries inside them in V1. If a report later needs, say, average render time, it can read `metrics->'render'->>'seconds'` without a schema change.

The scene plan's own `version` field (currently `1`) is the upgrade path: a future plan format bumps it, and `packages/shared` upgrades old plans when it reads them.

Data lifecycle and sizing

Rows are small and media is elsewhere, so a single creator stays far inside Supabase's free 500 MB database for years.

Data	Kept	Removed by
Projects, messages	Until the user deletes the project	`DELETE /api/projects/:id`, which cascades to messages, jobs, and video rows
Jobs	With their project	Cascade
Video rows	With their project; `deleted_at` marks the MP4 as gone from R2	Day-end cleanup sets `deleted_at`; the row stays so the library can say "expired"
`llm_calls`	90 days	Worker deletes older rows once a day
Heartbeat	Always one row	Never removed

Sizing estimate for an active creator making 5 videos a day:

Item	Per video	Per year
Project and plan	about 6 KB	about 11 MB
Job with frozen plan and metrics (1–3 renders)	about 12 KB	about 22 MB
Messages (brainstorm and revisions)	about 4 KB	about 7 MB
Video row and `llm_calls`	about 1 KB	about 2 MB
Total	about 23 KB	**about 42 MB**

That is under 10% of the free database even at that pace. The free tier's other limit that matters is pausing: Supabase pauses free projects after a week without activity, so a long break means pressing "Restore" in the dashboard before the app works again.

Migrations, setup, and backups

Schema changes are plain SQL files kept in the project's local Git history and applied with the Supabase CLI, so the database can always be rebuilt from scratch.

```
C:\projects\explainit\supabase\
  config.toml
  migrations\
    0001_init.sql        types, tables, indexes, trigger
    0002_rls.sql         row-level security policies
    0003_functions.sql   the nine functions and their grants
    0004_realtime.sql    publish jobs and videos to Realtime
  seed.sql               one sample project for local testing
```

```
-- 0004_realtime.sql: lets the browser follow its own jobs live (RLS still
applies)
alter publication supabase_realtime add table jobs, videos;
```

Setting up:

1. Create a free Supabase project (Mumbai region, the closest to Indian viewers and to the laptop worker) and turn on the Google and email sign-in providers.
2. Install the Supabase CLI, run `supabase link` with the project ref, then `supabase db push` to apply the migrations.

3. Copy the project URL and anon key into `.env` for the web app, and the service role key for the worker only.
4. Generate TypeScript types with `supabase gen types typescript` into `packages/shared/src/db.types.ts`, so queries are type-checked against the real schema.

Backups. The free tier has no point-in-time restore, so a weekly `supabase db dump` to `D:\ExplainIt\backups` keeps the schema and data safe. Videos need no backup: they are temporary by design, and any project can be rendered again from its saved plan and cached audio.